



Apache Kafka

Kodjo Klouvi | kodjo@osekoo.com



1 - Introduction to Apache Kafka

"A distributed platform for real-time data pipelines and streaming apps"



What is Apache Kafka?

- Apache Kafka is a **distributed messaging system**.
- Designed for **high-throughput, fault-tolerant, and scalable** real-time data streaming.
- Originally developed by LinkedIn, now part of the **Apache Software Foundation**.
- Kafka is like a **durable, distributed log**.



Why Kafka?

- **Kafka is ideal for:**
 - Collecting logs and metrics
 - Real-time analytics
 - Event-driven architectures
 - Streaming data pipelines



Why Kafka?

- **Key benefits:**
 - Horizontal scalability
 - High durability
 - Real-time streaming
 - Decoupling between producers and consumers



Kafka Use Cases

Domain	Use Case Example
E-commerce	Track user clicks and shopping events
Banking/Finance	Real-time fraud detection
IoT / Industry	Sensor stream ingestion and analysis
Web Applications	Logging and observability pipelines
Data Platforms	Ingestion layer before data lakes

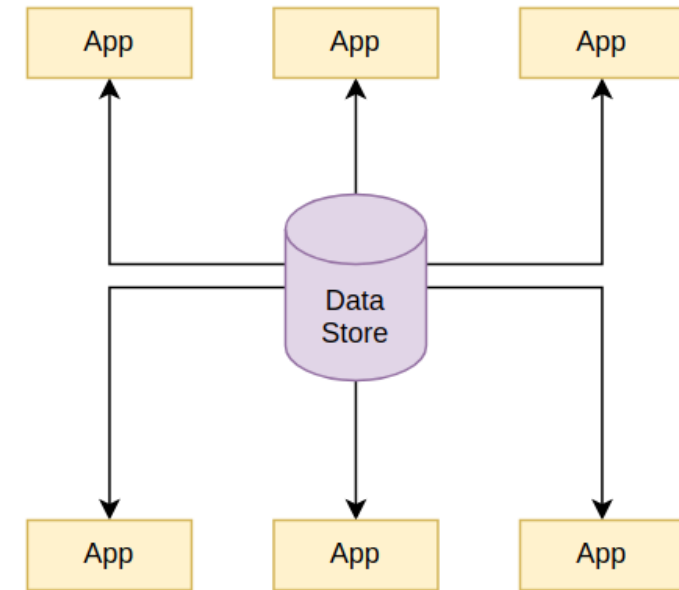


Kafka vs Traditional Messaging

Feature	Kafka	Traditional MQ (e.g RabbitMQ)
Storage	Disk-based log	In-memory (or short-lived)
Retention	Time-based	Message consumed then gone
Scalability	Very high	Limited
Performance	Excellent	Moderate
Consumers	Many per topic	Often 1:1

Traditional Data Access

- Multiple applications access the same data at the same time and apply the same transformations/operations
 - Risk of bottleneck
 - Massive resources consumption
 - Duplication
 - Heavy to maintain
- Synchronous actions (read - write)
 - Time consuming
 - Not parallelable





Kafka's Core Idea

- Kafka stores data as a **log** of immutable messages, organized into topics.
- Every message is:
 - Written by a **producer**
 - Stored in a partition of a **topic**
 - Read by consumers using **offsets**
- *"Write once, read many times – at your own pace."*

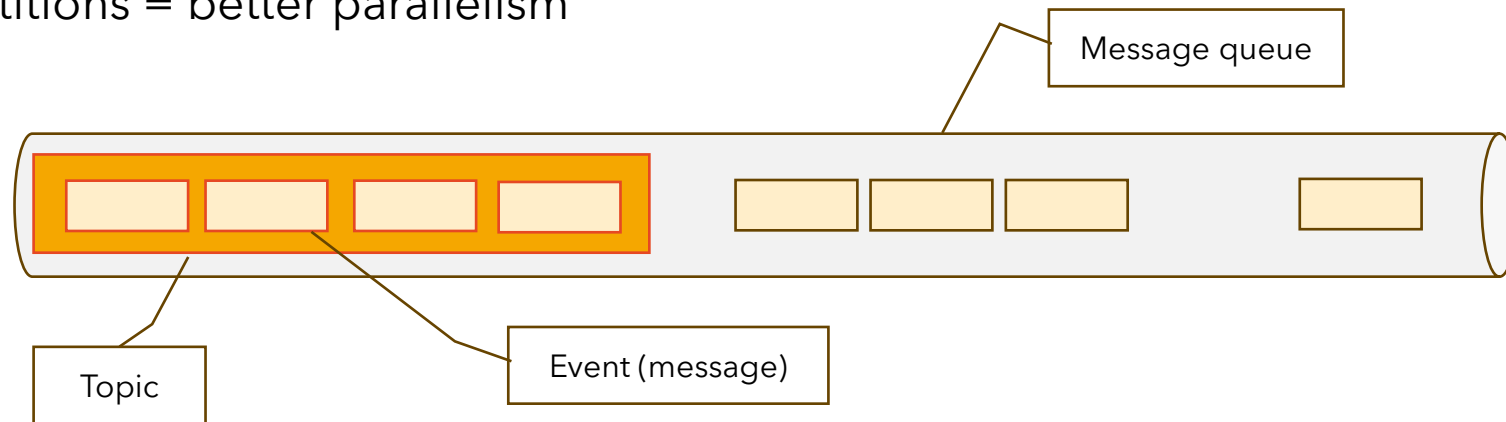


2- Kafka Core Concepts

"The building blocks of Apache Kafka"

Topics and Partitions

- **Topic:** A named stream of data (e.g. *orders*, *logs*)
- **Partition:** A shard of a topic (for parallelism)
- More partitions = better parallelism





Producers

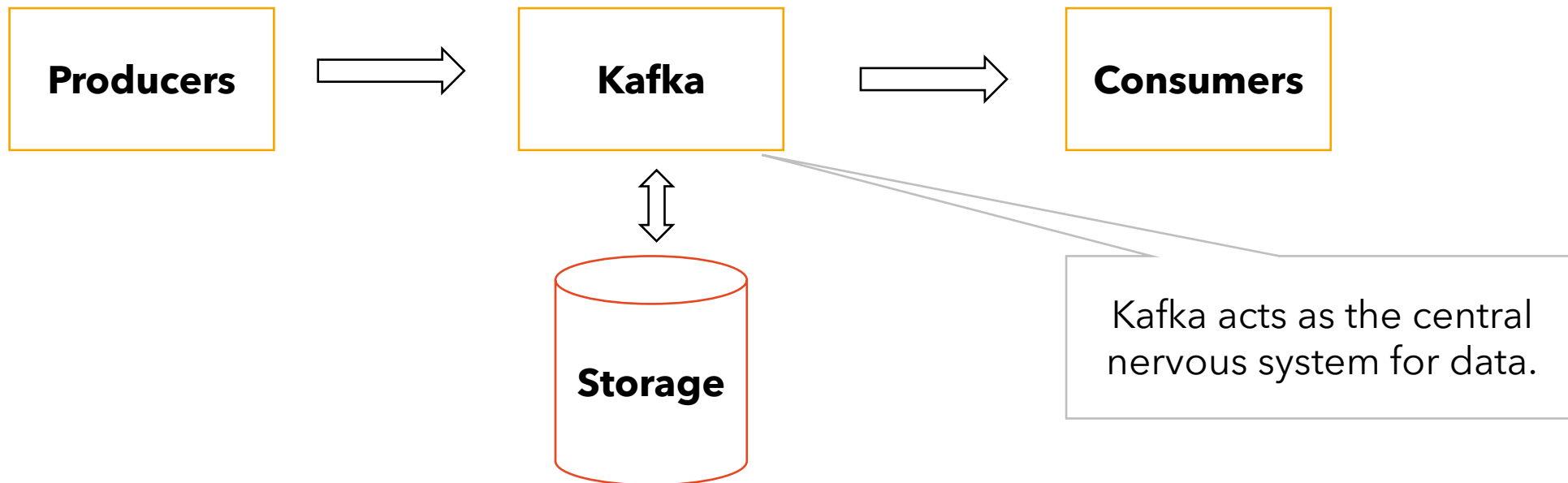
- Clients that **publish** records to Kafka topics
- Can:
 - Choose partitions explicitly
 - Let Kafka assign based on key (hash)
- Must handle:
 - **Retries**
 - **Acknowledgment levels** (*acks*)
 - **Batching**



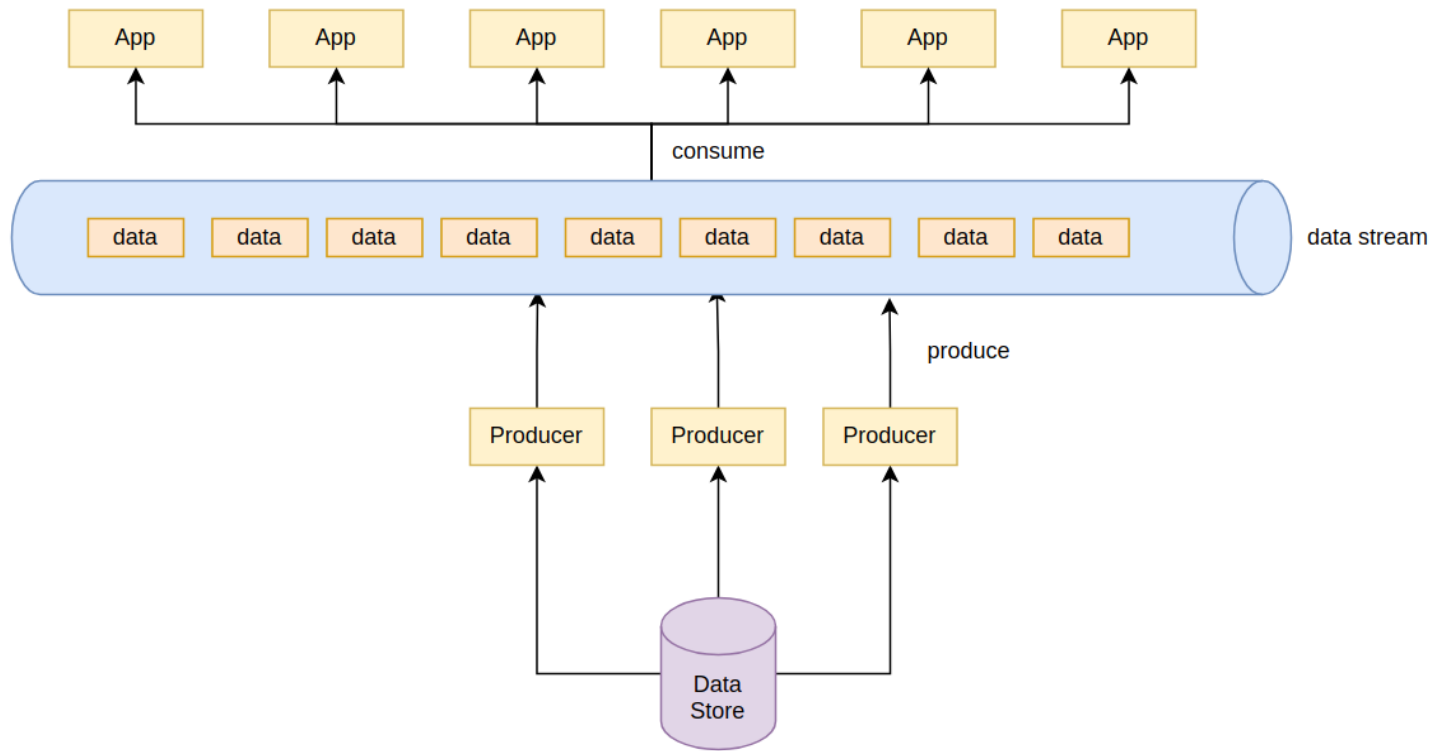
Consumers

- Clients that **read** records from Kafka topics
- Each record has an **offset** (position)
- Consumers **pull** data at their own pace
- Belong to a **consumer group** (for parallelism)

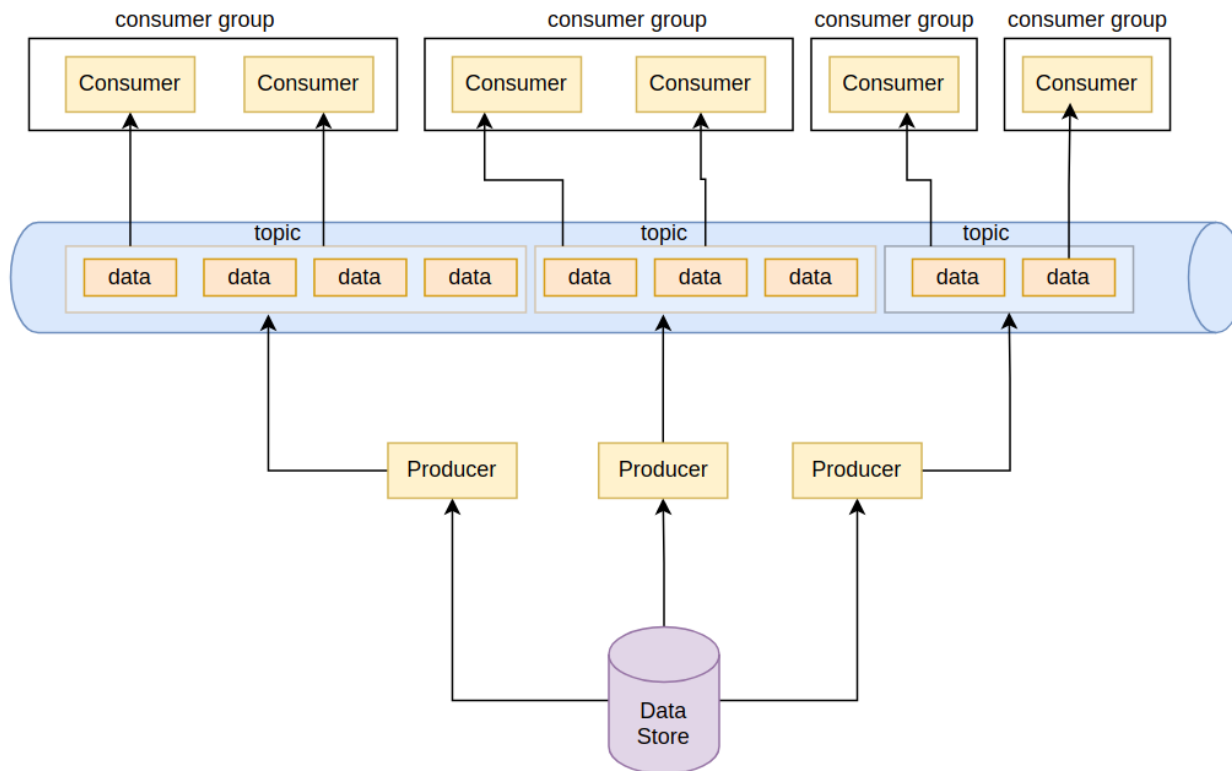
Kafka in a Data Architecture



Kafka in a Data Architecture



Kafka in a Data Architecture



Consumer Groups

Concept	Description
Consumer group	Set of consumers sharing the same group ID
Partition assignment	Each partition is read by one group member
Parallelism	Scales out reading across machines
Offset tracking	Managed per group

Guarantees horizontal scalability & fault-tolerance



Brokers

- **Kafka broker** = one Kafka server
- A **Kafka cluster** = one or more brokers
- Brokers:
 - Manage topics and partitions
 - Store messages on disk
 - Handle producers/consumers
- Brokers must be coordinated – done with **Zookeeper** or **KRaft (Kafka 2.8+)**



Offset and Retention

- Kafka stores messages with **offsets**
- Consumers use offsets to track progress
- Messages are **not deleted** after consumption
- **Retention options:**
 - **Time-based** (e.g., 7 days)
 - **Size-based** (e.g., 1GB per partition)
 - Or **manual deletion**
- Offsets can be **committed manually or automatically**



Summary of Concepts

Concept	Description
Topic	Logical name for a stream
Partition	Unit of parallelism
Offset	Message position in partition
Producer	Writes messages to topics
Consumer	Reads messages from topics
Broker	Kafka server
Consumer Group	Group of consumers sharing workload



Lab: Hands-on Kafka

<https://github.com/osekoo/hands-on-kafka>



3- Kafka Architecture Deep Dive

"How Kafka works under the hood"



Kafka Cluster Overview

- **Brokers** – Kafka servers (store and serve data)
- **Producers** – Publish messages to topics
- **Consumers** – Read messages from topics
- **Zookeeper** (or **KRaft**) – Coordinates brokers

A cluster can have **many brokers**, but each **partition has only one leader**.



Topic Replication

- To ensure fault-tolerance Kafka replicates partitions across brokers
- **Leader handles reads/writes**, followers replicate.

Topic: orders (3 partitions, replication factor 2)
Partition 0 → Broker 1 (leader), Broker 2 (follower)
Partition 1 → Broker 2 (leader), Broker 3 (follower)
Partition 2 → Broker 3 (leader), Broker 1 (follower)



Partitioning Strategy

- When a producer sends a message:
 - If a **key** is provided: Kafka hashes it → chooses partition
 - If no key: uses **round-robin**
- Why partitioning matters:
 - Parallelism
 - Order is preserved **within** a partition
 - Allows for **per-key processing**

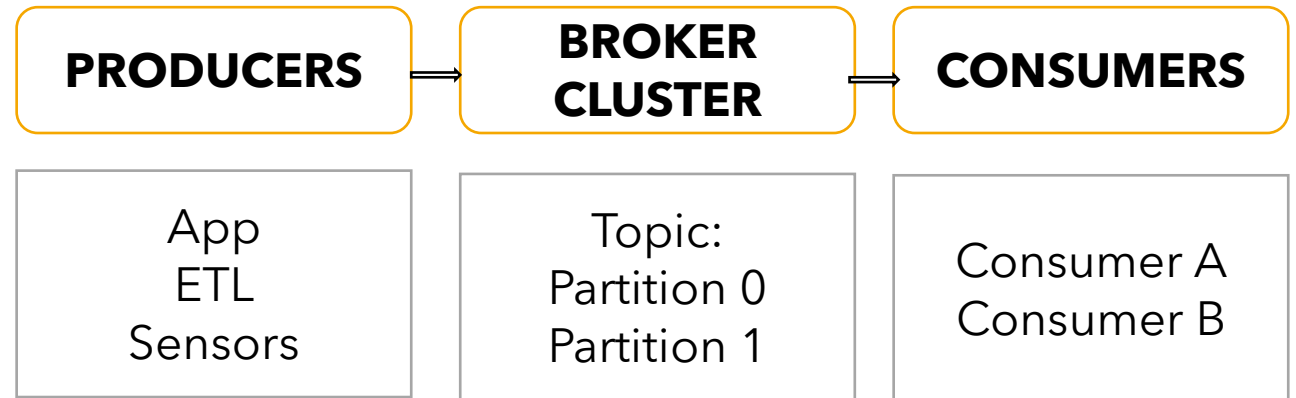


Leader Election

- Kafka automatically elects a **leader broker** for each partition.
- If a broker fails:
 - Zookeeper (or KRaft) triggers **new leader election**
 - Consumers and producers **automatically switch**
- Leader-follower sync is key to **high availability**.

Data Flow in Kafka

- **Producers write** to a partition
- **Brokers store** data with offsets
- **Consumers pull** data and manage offsets



Delivery Semantics

Semantic	Description
At most once	Fastest, may lose data
At least once	Guaranteed delivery, may get duplicates
Exactly once	Guaranteed no loss, no duplicates

- Spark & Kafka Streams support **exactly-once** with proper config
- Requires coordination with **transactional producer** and **offset commit**

Zookeeper vs KRaft

Feature	Zookeeper (Legacy)	Kraft (New Mode)
External coordination	Yes	No (self-managed metadata)
Cluster complexity	Higher	Lower
Default in latest Kafka	No	From Kafka 3.3+ (optional)
Production-ready	Yes	Since Kafka 3.x



4 – Kafka Producers and Consumers API

"Interacting with Kafka in code"



Kafka Producer: Core Concepts

- A **producer** sends records to a Kafka topic.
- Producers are **asynchronous** by default.
- Configurable options:
 - `bootstrap.servers` – list of Kafka brokers
 - `key.serializer` and `value.serializer`
 - `acks` – how many replicas must acknowledge a message
 - `linger.ms`, `batch.size`, `retries` – for performance tuning

Kafka Producer: Example (Python)

```
from kafka import KafkaProducer
import json

producer = KafkaProducer(
    bootstrap_servers='localhost:9092',
    key_serializer=str.encode,
    value_serializer=lambda v: json.dumps(v).encode('utf-8')
)

data = {"user": "alice", "action": "click"}
producer.send(topic="events", key="user1", value=data)

producer.flush()
producer.close()
```




Kafka Consumer: Core Concepts

- A **consumer** reads records from a topic (or topics).
- Consumers **pull** data – Kafka does not push.
- Key configs:
 - `group.id` – required to join a consumer group
 - `enable.auto.commit` – auto-offset management
 - `auto.offset.reset` – where to start if no offset found (earliest, latest)
 - `key.deserializer`, `value.deserializer`

Kafka Consumer: Example (Python)

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    *topics: "events",
    bootstrap_servers="localhost:9092",
    group_id="analytics-group",
    auto_offset_reset="earliest",
    enable_auto_commit=True,
    key_deserializer=bytes.decode,
    value_deserializer=lambda v: json.loads(v.decode("utf-8"))
)

for msg in consumer:
    print(f"[{msg.key}] {msg.value}")
```



Manual Offset Management

- Default = **auto-commit** every 5 seconds
- For more control, disable and manage manually
- Use manual commits for exactly-once processing logic.

Manual Offset Commit (Python)

- For **exactly-once** processing, **disable auto-commit**
- Use `.commit()` after successful processing only

```
from kafka import KafkaConsumer
import json

consumer = KafkaConsumer(
    *topics: "events",
    bootstrap_servers="localhost:9092",
    group_id="analytics-group",
    auto_offset_reset="earliest",
    enable_auto_commit=False,
    key_deserializer=bytes.decode,
    value_deserializer=lambda v: json.loads(v.decode("utf-8"))
)

for msg in consumer:
    print(f"[{msg.key}] {msg.value}")
    consumer.commit()
```



Consumer Group Coordination

- Kafka distributes partitions to consumers in a **group**
- If a consumer crashes:
 - Kafka **reassigns** its partitions
 - Offsets are preserved
- Consumers are **scalable** and **fault-tolerant**
- Use **one consumer per thread** for best results.



Consumer Groups & Parallelism

- Multiple consumers in a **group** share partitions
- Kafka rebalances partitions on:
 - Consumer join/leave
 - Topic changes
- Rebalancing is **automatic**



5 – Kafka Operations and Best Practices

"Deploying, securing, and managing Kafka in production"



Retention Policies

- Kafka retains messages even after they are consumed.
- You can **tune retention** per topic using CLI or API.

Config	Description
retention.ms	Keep messages X milliseconds
retention.bytes	Keep max X bytes per partition
delete.retention.ms	Time before compaction applies

Topic Management

- Create topics manually (or auto-create if enabled)
- Best practices:
 - Match **partitions to consumer parallelism**
 - Use **replication factor ≥ 2** for durability
 - Use consistent **naming conventions**

```
kafka-topics.sh --create --topic events \  
  --bootstrap-server localhost:9092 \  
  --partitions 3 --replication-factor 2|
```



Partition and Replication Strategy

- Partitioning is key for:
 - **Scalability** (parallel consumers)
 - **Message ordering per key**
- Replication provides:
 - **High availability** in case of broker failure
- Monitor ISR (in-sync replicas) to detect lagging followers.



Security in Kafka

- Kafka supports several layers of security
- Recommended:
 - Use **SSL** for encryption
 - Use **SASL SCRAM** for username/password auth
 - Restrict **producers/consumers via ACLs**

Layer	Options
Encryption	SSL/TLS
Authentication	SASL (SCRAM, GSSAPI, OAUTH)
Authorization	ACLS per topic, group or user



Backup and Disaster Recovery

- Kafka itself is durable, but disaster recovery requires:
 - **Replicated clusters across regions**
 - **MirrorMaker 2** (to replicate topics between clusters)
 - **Snapshotting** logs (filesystem backup)
- Consider tiered storage or retention policies to reduce disk pressure.



Operational Best Practices

- **Checklist for production readiness:**
 - Tune `num.partitions` per topic
 - Enable and monitor JMX metrics
 - Set `retention.ms` per topic
 - Configure `acks=all` in producer
 - Secure Kafka with TLS + SASL + ACLs
 - Monitor consumer lag
 - Use MirrorMaker 2 for DR/multi-DC setup



Lab: Hands-on Streaming

<https://github.com/osekoo/hands-on-spark-streaming>